Introduction & Recap of Self-Attention

And if you remember, what self-attention is that each token is attending to all other tokens in the sequence through this mechanism of attention. So you have these notations of queries, keys, and values. So here the idea is that the query is going to ask which other tokens are most similar to itself by comparing query and key. And then once that's done, basically we will be taking the associated value. So we saw that the self-attention mechanism can be expressed with this formula, so softmax of query times key transpose over square root of dk times v. So I hope this formula is familiar for you. So just know that this formula is highly optimized. These are big matrix multiplications, or hardware, is very capable of doing, it's very optimized in doing that. And all of that to say that we also introduced the architecture of the transformer, which you can see on the right. So here, if you remember, the transformer is composed of two main components. So the encoder on the left side and the decoder on the right side. And the transformer was initially introduced in the context of machine translation. So you can think of the left side as processing the input text in the source language, say English. And the right side is responsible for decoding the translation in a target language, let's say in French.

And this multi-head attention layer is where the self-attention mechanism happens. And I remember there was a lot of questions regarding-- it's called multi-head attention layer. So there are several heads. What does that correspond to? So in the transformer paper, which is the "Attention is All You Need" paper, you have this figure, which actually represents each of these heads. And you can think of each head as an opportunity for the model to learn one way of projecting the input into being a query, a key, or a value. So just being a bit more clear, so for instance, for the query and the key, so this is like each head. So the number of these little boxes is basically the number of heads. And to better visualize and understand what this means, I also wanted to show I guess what is being shown in the paper, which is a way to interpret what each of these heads do. So we have this concept called attention map, which basically tries to represent the value of each of these query, dot product query.

So in this example, what we're interested in is to see which other token is being most similar to token its. And so what we do is we take a look at the quantities,

the query that is representing its, dot product, all other keys. And we look at what are the keys are leading to a high value of the dot product query times key. And when you do that, basically what you see or what the paper sees is you have these two words, so application and law, which are highlighted as being with the high-attention weight. So attention weight here being the dot product of the query its, and the key for each of these tokens. And I guess there is also a way to interpret those. And here you can see that the tokens that are being highlighted are law and application, which basically makes sense because the token its is referring to law. So basically, the model needs to learn how to associate these words with what happened before. And its is also referring to application, which is also another way of explaining why that is the case.

And so here what the authors chose to do was to show these values as a function of these different heads. So for instance, on the left side, these are the intensity for heads, for let's say the first heads. And then the second heads shows that the intensity is very high for law. So basically, long story short, these heads may learn, I guess, different ways of figuring out what words matters. Yeah. Great question. So the question is, when we're doing all these computations, are they going through different MLPs. The answer is that we're going to have different projection matrices for each of them. So we had this detailed example that actually went through that where each head is going to have its own projection. And in parallel, you're going to have that computation that's going to happen. So each head is going to have one result here, that is then going to be concatenated and then projected once again when the output matrix. So yeah, long story short, it's highly parallelized. And it's basically just like projections. And here you have some matrix multiplication and softmax. Does that make sense? Cool. Any other questions on this? Cool.

So I guess this is just a way to illustrate the conversation we had for lecture 1. There was some questions about these attention heads, what they do. And I guess looking at the attention maps is one way of making sense of what they mean. Cool. With that, I highly recommend that you read the transformers paper, so "Attention is All You Need." So it's a very dense paper. It's just a few pages long. But I hope that what you've seen in lecture 1, you'll be able to digest the content in a way that will make sense to you. Cool. So with that, we're going to start the actual meat of what we're going to discuss today. So surprisingly, this transformer architecture, which was introduced in 2017, is

actually an architecture that has still stayed relevant along the years. And there are a few components that have slightly changed. And we're going to see which ones they are. So there are some slight variations. But overall, today's models are, we're going to see, all more or less based on the initial transformer architecture. So we're going to try to divide the class in two parts. So the first one is what I'm going to cover, which is what are the parts of the transformer that are important and that had some variation. And in the second part, Shervine is going to talk about, I guess, the nomenclature of today's models and how they relate to the original transformer. Cool. OK, so let's start with the first important concept that's in this architecture.

Overview of Position Embeddings

And this is the position embedding. So if you remember, here we're letting tokens interact with all other tokens in a direct fashion. So they have direct links. But contrary to things like RNNs, where you have a sequential dependency where you process each token one at a time, here you're basically losing this idea of a token being processed before another one. So you lose this position information. So as a result of that, we need to somehow quantify tokens at each position and try to inject that information when the transformer is processing the inputs. So how are we going to do that? So the original transformer paper authors, they chose to have a dedicated embedding. And when I say dedicated, what that means is each position has one embedding. So position 1 has one embedding, position 2 has one embedding, et cetera, et cetera. And what they chose to do is to add that embedding to the input-token embedding. So for instance, if I say a cute teddy bear is reading, which is position number 1, representing the token a, plus the embedding representing the first position.

Yeah. It's a great question. So the question is, are the position embeddings learned or static? Both, as in the authors have tried both. And we're going to see what the second one is. But I guess here let's suppose that they are learned. So what does that mean? So that means that basically you need to learn embeddings for each position. And the problem with this approach is that you're very much dependent on what is in your training set. So for instance, like here, if you have somehow a text that always has something that is happening at position number 2, your learned embeddings will have that bias kind of learned. So that's one limitation of that. Second limitation is you can

only learn positions up to the max number of position that is in your training set. So let's suppose you train your transformer on sequences that are up to, let's say, 512, let's say. You can only learn position embeddings up to that position. Right?

Yeah. So the question is, I guess, how do you parameterize that? So I guess what you do is you have a placeholder of a learnable position embedding, let's say between position 1 and 512. And basically, when you do your training, you're just letting these weights be learned through the regular gradient descent, all these things. So yeah, so this is like the first method. But as I was mentioning, it has its limitations, because you can only learn embeddings of positions up to the max position that is present in the training set. So for instance, if you have at inference time a position that is beyond the position that was in the training set, well, you have not learned that. So you need to find a way to infer the value. So that's the second limitation. But yeah, but on the pro side, I guess you're just letting your model learn. And we've seen that the gradient descent does wonders when it comes to just learning from the data. So yeah. And for these reasons, these methods was something that the authors said that was performing well, along with the second method, which is different, which is around having an arbitrary formula for each dimension corresponding to a position embedding. And we're going to see that now.

Sinusoidal Embeddings

So first method was you have one embedding per position, and you just learn that. Second method is you have one embedding per position, but you're not going to learn that. You're going to have something that is predetermined that you're going to use. And we're going to see that what the authors choose was a formulation using sine and cosine. So it can feel weird, why did they choose this. But we're going to see why that makes sense. So the idea here is for a given position, let's say m, have a vector of size d model. So d needs to match the dimension of your token embeddings, because of course you're adding them. And what you're going to do is, for every index you're going to compute the corresponding value with respect to these formulas. So what are these formulas? So it's basically sine of something times m. And we're going to see what that something means. And then the second one is cosine of something times m.

So who remembers trigonometry formulas? Cool. Everyone. So before we go into that, let's just simplify the notations. Let's just assume that this big quantity that you saw is actually something like omega. So let's suppose it's omega as a function of i times m. And you note omega i as being this quantity, so 10,000 to the power of minus 2i over d model. Let's suppose you construct your embeddings to have this way. Then, I guess I want us to think about why that would make sense. Because if you think about it, what you want is to represent positions in a way that reflects the following fact-- words that are close together are likely to be more relevant, as opposed to words that are further together. So if you have two words that are like just one position apart versus 10,000 position apart, what you want is that the one that is one position apart is more similar than the other one.

So let's see if the formula makes sense. So let's suppose you have two position embeddings, so one at position m and the other one at position n. And let's suppose you compute in all the values from this predetermined formula. Well, it turns out, if you remember your trigonometry formulas, so cosine of a minus b is equal to cosine of a cosine of b, plus sine a sine b. Right? Well, turns out that if you express cosine of omega i, factor of m minus n, this is something that you obtain. It's just the identity that I mentioned. Well, it just turns out that this quantity is just one component that appears when you do the dot product of these two position embeddings. Right? Because basically here, when you do the dot product of position m and position n, what you do is you take the first position, you multiply them, then you plus the second position, you multiply them, et cetera, et cetera. And then you come here, it's sign of this times sine of this, plus cosine of this times cosine of this, which is just this quantity. So at the end of the day, what you realize is that when you perform a dot product of these embeddings, you end up with a sum of cosine that are a function of the relative distance between m and n.

Yep. What do you mean by pair, by the way? Exactly, yeah. So the question is, so the closer they are, the more similar they are. So that's the intuition, that basically this way of formulating the embeddings is trying to approximate or it's trying to mimic. So with that, you basically obtain a dot product that is just a function of m and n, the relative distance between them, actually. And just as a reminder, so why do I care about the dot product? Because if you remember, in the embedding world when we try to quantify the similarity between two embeddings, what we do is typically something that is involving the dot product

of these two. So you typically have the cosine similarity. The cosine similarity is just dot product over the norm of each embedding. So which is basically a dot product. Right? So that's why we care about the dot product. And here we see, great, it's a function of the relative distance between the two. So in particular, again, if you remember your trigonometry class, cosine of 0 is 1. And the higher this number, the lower the value of cosine of this number. Of course, it's periodic, so what I'm saying is not necessarily true. Past pi, just goes the other way. But I guess what I'm trying to say is for m equal to n, you have basically a sum of cosine of 0. And it is the value at which this quantity is maximum. So when m is equal to n, this quantity is maximum. Which means basically if you're looking at the position itself, it's the most similar. Basically, it matches our intuition.

And so now when you plot the values of the embeddings, this is what you obtain. So here in this graph on the y-axis, I'm basically representing all the embeddings for each of the, let's say, 50 positions. And on the x-axis, it's basically values along a given vector across several dimensions of a vector. So if you take, let's say, the first row, you're looking at the first or number 0 position, depending on how you index your vector. And you're looking at all the values of the position 0 embedding. And so you see that for low dimensions, this value goes up and down very frequently. So it's more high frequency. And when the dimension is high, it basically takes a lot of time for the value to go up and down. So it's basically more low frequency. So this relates to this omega i that I mentioned, this omega i. So omega i is very high for low values of i, which is the dimension. And it's very low for high values of i. So this basically just determines how quickly your cosine and sine basically vary.

Cool. So this is what the original authors have tried. And basically what they said, what they noted was that using these methods leads to comparable results compared to the learned one. But here we have a big advantage, because it can extend to any sequence length, not just a sequence length that you saw at training time. And this is one of the reasons why this may be something that is preferable. So yeah, this is the intuition and this is what the authors chose. Now, fast forward to 2025. I guess you may ask me, are we still using that? And the answer is kind of. So we're still using this idea of we want far-- I guess, tokens to be less similar than closer tokens. But we're not injecting the embedding like they did. And we're going to see why.

T5 Bias, ALiBi

Because if you remember, what you care about is determining how similar tokens are in the self-attention computation. And where does the self-attention computation happen? In the attention layer. But here, what did I say? I said, let's compute these embeddings and let's add them here. But actually what we want is to reflect this similarity in the attention layer. Do you have a question? Yeah. So in this first method, yes. So the question is, is it added to the input feature. Yes. Yes, just add. Yeah, yeah. But the problem is, we mostly want these, I guess, intuition to hold true in the multi-head attention layer. So this is one of the reasons why people have tried different variations, and in particular have these position embeddings intervene directly in the attention layer, as opposed to the input. Because basically when you do it at the input-- just here, fair, OK-- it's going to be roughly something that is going to go into this attention layer, but it's kind of indirect. So what we want is to directly do something about the attention formula that would, I guess, reflect the fact that we want close tokens to be more similar, compared to further tokens.

And the way we do that is, if you remember, the self-attention layer is basically the softmax of qk transpose over square root of d times v. So what we want is to add a little something inside that softmax, which is basically where you quantify how similar a token is to another token. You want to add a little something to reflect the fact that some tokens, they're supposed to be more similar compared to that token, compared to others. So there's a few methods that have tried to have some variation of that. So for those of us who know the T5 paper, what we're going to see a little bit later, they have tried these relative position bias by learning the bias term, which is in the formula above. So what they did was, let's suppose you have a given distance between the positions m and n. So their idea was, let's learn that. Let's basically bucketize all m minus n into some buckets. And let's just have the model learn these quantities that are then going to be injected into softmax.

Yeah. So the question is, does that pose a problem that the bias is here with respect to the probability at the end? Because it has to sum to 1. Well, you can do whatever you want inside the softmax, because the softmax is going to normalize it anyways. So you can think of the bias as being something that is maybe more negative for things that are far apart, compared to things that are closer together. So T5 says, let's learn them. We have another message from, I

guess, this "Train Short, Test Long" paper, which introduced this method called ALiBi. ALiBi stands for Attention with linear bias, I believe. And what they did was, say instead of learning those biases, let's actually have a deterministic formula, which is as a function of the relative difference between those two positions. And they said that. So they had some results. So all these papers, they always compare based on one another to see which one is, I guess, more performant. But the reality is is that in today's models, most models actually use another kind of position embedding method. And we're going to see it now.

RoPE (Rotary Position Embeddings)

So this method relies on rotating the query and the key vector by some angle. And I guess you can think of it as, you have your query, you have your key, let's suppose in a 2D space. So what you're going to do is rotate your query by some angle that is a function of its position. And you're going to rotate the key vector by some angle that is a function of its position n. And I guess how do you do that, by the way? I wasn't supposed to show this thing. So let's suppose you have a vector, by the way. And you want to rotate that vector, because I had the answer on the slide. But how would you go about this, I guess, for people who just want to talk about this with intuition? So who here has done, I don't know, rotations in space? Yeah. That's correct, yeah. Matrix multiplication. And you're going to use a quantity that's called rotation matrix. And the rotation matrix is expressed as follows. So it's basically a 2 by 2 matrix in the 2D plane that has cosine of this angle, minus sine of this angle, sine of this angle, cosine of this angle. I'm looking at the time. It's quite simple to just show that it works, but we may run out of time. Do you want me to quickly show you that it's indeed a way to rotate the vector? OK.

So here as a reminder what we're trying to show is that we can use a matrix multiplication to rotate a vector in 2D space. So let's suppose we have the following vector. That is something that you can quantify with two dimensions, let's say x and y. You can express your vector in 2D space with this. Right? But I guess this, if you note, are the norm of the vector, and phi, the angle with respect to the x-axis. You can also write v as R with vector cosine of phi and sine of phi. Right? So if you multiply the rotation matrix with this phi, what you're going to obtain is some multiplication of cosine minus sine, sine and cosine of this and that. And I will leave this exercise for you. But you can show that rotation times this v can be expressed as R of cosine of theta plus phi, and

sine of theta plus phi. So this is a quick proof. So I'll just leave you the multiplication of the rotation matrix and v. But you will obtain these trigonometric identities that will, I guess, lead you to this formula. This basically shows that if you multiply this matrix and this vector, you're basically rotating the vector by this angle.

Yeah. So the question is, why do you want to do this? It's a great question. It's my next slide. So this is just a little intro. So I guess here, just going back to this method, what we want to do is to quantify the similarity between tokens, and have close tokens be more similar compared to tokens that are more afar. So the problem that we had with the previous methods was-- so in the first method, this learned embedding, you always had this overfitting issue. Because basically when you learn these biases, it always depends on which training set you have. So maybe your data set is in a way that, let's say, tokens that are close are similar, but in a different way compared to what you see at inference time. And this ALiBi method, it didn't have that learnable component, but it was quite restrictive, because it's a very simple formula after all, just the relative difference between n and m. So I guess people have tried different ways of coming up with something that tells you that, I guess, an embedding that reflects the fact that you want further positions to be less similar than closer ones.

So in this method, we're going back to the sine and cosine world from what the author had proposed. And think about similarity from the lens of cosine and sine functions. So this is a little intro. And their method is called RoPE. So I'm not sure if you've heard of it. So it stands for Rotary Position Embeddings. And we're going to see that this method-- so why do we care about this method? So this method has two great things. So the first one is that if you rotate the query and the key, you will end up with a quantity that will be a function of the relative distance between the two. That's going to be very nice, and that's why I wrote this thing on the blackboard. Not sure if you can see, by the way, but we won't have time to go into the mathematical detail. But if you want to just express these things at home, this is just the foundation. And so in particular, if you remember your attention formula, so you have query times key transpose. So if you rotate the query by an angle m and the key by an angle n, what you're going to end up is a formula that has the rotation matrix of angle theta and, I guess, n minus m. And this is great because this is a function of the relative distance between these two positions.

OK, so why do I talk about this in detail? Well, it turns out that most models these days, they use RoPE, which is why it's important. And I would say another thing, it's maybe a little bit hard to get the intuition as to why that works. But hopefully, the explanation that I gave regarding the sine and cosine at the very beginning can help you just build that intuition. And speaking of that, it turns out that the upper bound of the attention weight given by the query and the key is such that we observe a long-term decay. Meaning that as n minus m is large, we do see the upper bound that gets smaller and smaller. Well, you see these little oscillations, it's not perfect either. But we do have some mathematical, I guess, results as to just the upper bounds decaying over the long-term.

Cool. Any questions on this? Yeah, yeah. Exactly, yeah. So the question is the relative distance is captured in the rotation matrix-- and yes, yes. Oh, yeah. It's a great question. So the question is, what is theta? So theta is actually fixed. So do you remember this omega that I talked about here? Here. So it's basically some function of i and d. I actually pass it quite quickly. But what I showed you is in the 2D space. But here we're in a d-dimensional space, which is greater than 2. So I glossed it very quickly. But the way you extend this method is by having these 2D space by block. But then the theta is a function of typically something that you fix, but a function of i, which is the dimension. It's basically between 1 and d over 2. It's a function of that, and it's a function of d as well. You will see this theta as being roughly equal to omega i, just this one, more or less, more or less. Sorry? Oh, so the question is, so that it has the same dimension as the latent dimension? So, well, here you have a product of matrices. So you need to have the dimensions match. So I guess your answer-- yeah. Does that make sense? I'll take that as a yes. So I spent a bunch of time on this, because I think this is actually quite important. A lot of models use this. And yeah, I guess the intuition is not super obvious. So I hope this was helpful. So this was position embeddings. Yeah. Oh, yeah. Yeah, so the question is about, how do you obtain this curve? So this is actually a curve that I believe is shown mathematically. So it's kind of complicated. We're not going to write down the formula. But if you're interested, so in this paper, in the RoFormer paper, there's an appendix where they show mathematically that it is upper bounded by some quantity. And this is what this is showing. Yeah, great question. Cool.

So, position embeddings is one part of the transformer that has changed a little bit. And we've seen how that changed, and why. So now we're going to see another component of the transformer that has also a little bit changed.

Layer Normalization

And that component is the layer normalization. So if you remember, the transformer architecture, which is composed, again, of encoder, decoder, and then you have the components inside. So you have these boxes that say add and norm. So what do they mean? So basically, here, what we do is we take the input, as well as the output of this sublayer. We add them together, and then we normalize. So this is a little trick that the authors do. And it is shown in practice to improve convergence and just make the convergence be quicker. So the idea is as follows-- if you have a vector, sometimes the components of the vector can be super large, sometimes they can be super small. The idea here is to normalize the components of your vector within some range, some normalized range. So the way you're going to do that is you're going to take your vector, and then subtract it by the computed mean, which is basically the sum of its components, and normalize it by basically its standard deviation. And what you're going to do is you're going to learn two quantities-- one, gamma, which is going to be the rescaling factor, and then beta, which is another term as well that you learn. And you're going to let these two quantities be learned by your model. And so in practice, as I mentioned, so what this does is it helps with training stability and with convergence time.

So this was a technique that was used in the original transformer paper. I just want to call out that there has been some changes since then. So we went from normalizing the input plus the output of the sublayer, to having a sum of the input and the sublayer of the normalized input. So in other words, what we've done is to change where the normalization is located. So here in the transformer paper, it was what we now call a post-norm version. And nowadays, we use a pre-norm version, which basically consists of having the LayerNorm right before the vector goes into the sublayer. And sublayer here can be either the attention layer or the FFN. But not only that, there is also another change. So nowadays, people, they do not use LayerNorm. They use something else called RMSNorm-- RMS, Root Mean Square, Normalization, which is basically a variation of what you've seen before. So instead of computing this, basically what people do is they just normalize x by the root

mean square of the components of x, and they learn gamma, only gamma. So why do they do that? Basically, they show that the convergence properties, they're basically comparable. But here you have fewer parameters to learn. So it's basically quicker.

Yeah. Good question. So the question is, what is the intuition behind normalizing? So the intuition is that, if you look at your model, you have several layers. In some layers your vector-- your activation, to be more precise-- so do you know the vector that you see that goes from here to here is basically called activation. Sometimes the activation has extreme values in one part of its component, sometimes in another part. And the model is typically having trouble in learning the weights in each of these layers if these activations, they vary too much. So the idea is to bring the values of the components of the activation to some range that is not too far off in some direction. So in case you're interested, there is this key word, internal covariate shift, which is basically the term that is given to the phenomenon that I'm describing here. So yeah, that's the intuition.

Yeah. Oh, great, great question. So question is, what is the difference between this and batch normalization? So batch normalization is normalization across the other dimension, which is the dimension of the batch. So let's suppose you have a bunch of vectors. What you do is you normalize each component with respect to all the other components for the same dimension, but of the other vectors. So you can think of it as just another way of normalizing. But that being said, when it comes to these transformer-based models, typically people use LayerNorm, probably because empirically it works better. But also because BatchNorm, you're basically also dependent on the batch. And it can introduce, I guess, differences between training and then inference. So that's basically the reason why. Cool, great.

So we've seen position embeddings. We've seen layer normalization. So now we're going to see a third important component of the transformer, which is the attention.

Sparse Attention

And in particular, I guess there is something I've not really emphasized. But when you do self-attention, you're basically letting every token interact with all

other tokens. So when you look at, let's, say a matrix that shows all the interactions, so you have n, the sequence length. And the sequence length, you basically have o of n squared complexity, which is a lot, especially as n grows longer. So people have tried to approximate this o of n squared into something that is a little bit more tractable, but does not lose the performance. So there is this paper in 2020 that came out, "LongFormer." So what it did was just try different versions of the attention by restricting the window at which it operates. So here, instead of letting each token interact with everyone, each token only interacts with its neighborhood.

Yeah. Oh, again, a great question. So question is, do you do that after the attention matrix computation? You mean the softmax, right, softmax? So you raise a great point, which is, if you do the softmax of everything, why would you do this? So in practice there is a bunch of implementations that does a clever-- so it's called tiling-- a bunch of clever operations that do not involve this huge matrix operation that you see in the softmax. Again, I guess, like against the softmax of qk transpose over d. You're not going to compute the whole thing. You're going to have some cleverness in how you compute that, basically. Yeah, so the question is, can it be something that's comparable to convolutions? And we're going to see that in a second. But you have some similarities with the vision world. And we're going to see that in a second.

Cool. So nowadays when you have local attention like this, like this, people use the term sliding window attention. So when they use that term, they mean this, which is basically just restricting the attention to neighboring tokens. And nowadays what people do is in some layers they will have local attention. In some others, they will have global attention. And they interleave these layers. So depending on the model, they typically try different combinations. So there's not a set recipe. But it's typically something that is used nowadays. So the window here, I mean, for, illustrative, purposes, the window is super small in my illustration. But I guess nowadays with the sequence lengths that can be very big, you can think of this window as being, o of several, thousands. And just to give you another example. So back to the convolution comparison that you mentioned. So you have some architectures. And here I'm going to take the example of a mistral that has these sliding window attention at every layer. But then when you think about it, the token here can attend up to the token here, but then the token here can attend up to the token here, et cetera, et cetera. So I guess, if you think about it, it's similar to the idea of the receptive field in

computer vision. So I'm not sure if you're familiar or from the computer vision world. But if you are, what this means is just taking one token and trying to think what other tokens has this token effectively interacted with, which is basically the question that people also sometimes ask. When they do convolution, they're like, OK, so this value, what are the values did it actually see? So you can also think it this way.

Sharing Attention Heads

OK, so first variation is, instead of doing the full n by n attention, people sometimes do local attention. The second variation that is orthogonal to all of this is to not have one projection matrix per head, but to share projection matrices across heads. So here the idea is you have, let's say, h heads. The idea is you're going to have some number of projection matrices for the query. But then what you're going to do is to group the projection matrices for the key and the value that you're going to share across several heads. Now you may ask, why do you share projection matrices for the keys and the values, but not for the queries? Is this the question that you're wondering?

Well here, I guess, people do things to just try out if something works or not. I guess here you can intuitively think that the query is basically wondering if something is similar to another thing. So I guess you can ask yourself this question in different ways. So it may make sense to keep that diversity. But one of the core reasons why we choose to group projection matrices for the keys and the values and not the query is because, when you decode, you perform attention between the current word and all the words before. So in other words, every time you generate a new word, you're going to attend that word to all the words before. So the keys and the values, they're going to come up a lot. So every time you want to decode something, you need to attend to all other things, again and again. So we're going to see in, I think, next, lecture there's something called the KV cache, which basically saves the values of the keys and values. And one thing that we want is for that cache to not become too big. So if you share projection matrices across heads, just allows you to save a little bit of space, a little bit of memory. That's the why. Does that roughly make sense? OK, cool.

So speaking of that, there are some variations as to, I guess, just how many projection matrices to share. So you have the extreme example where all h

heads, they all share the same projection matrices for v and k. And in that case, this method is called MQA, Multi-Query Attention. You have the in-between case where you share g, where g is the number of groups g projection matrices for v and k. And then you have groups of size, let's say, h over g. So this one is called Group Query Attention, GQA. And then you have the case that you are very familiar with from the transformer, which is every head has its own query projection, key projection, and value projection matrices. And this is the standard, multi-head attention.

Cool. So I'm, going to just take a pause to see if what I mentioned makes sense. I'm going to soon transition it to Shervine. But before I do, I just want to make sure that what I discussed is making sense, roughly. Yeah. So the question is, do we apply this to self-attention and cross-attention? I don't want to spoil the show. So I'm going to talk about something later on. But where is my trans? So if you take a look at the transformer architecture, we're going to see soon that what we care about most is what is in the decoder, because we actually forego of the encoder in nowadays' models. We have not seen this yet, but I'm just telling you. And so this typically comes into play for the masked self-attention in the decoder. But this technique can be applied in all attention layers. But I guess what I'm telling you is modern LLMs, their decoder only models which basically only have the decoder part of the transformer. We're going to see this in a second, which is the masked self-attention. So I'm not going to say too much because Shervine is going to cover that. But yeah, just a quick peek of what we're going to see.

Cool, yeah. Yeah. Great question. So the question is, when do you know which one you should use? I would say the choice is always driven by a few factors. One is how well it performs. Second is how much do you care about things like latency costs. And so it really depends how big is your model, how much you want to save on, let's say, compute, how is your input length, for instance if you have a shorter input length. So here what we want to do is to avoid having to do all these things for the whole o of n squared thing. So I guess all of these come into play. So I would say it's not straightforward as to what the answer should be. But I would say a lot of recent models, they tend to share projection matrices. So typically, I would say GQA is what you would see. But it's not necessarily the case for all models. Cool. We're running out of time. So I'm going to have Shervine here for the second part. Thank you.

Transformer-Based Models

OK, great. Thank you, Afshine. So we're going to continue this lecture with some deeper dive into the kinds of models that we have in the transformer landscape. And then we're going to do a deep dive into one specific architecture that is very useful for classification settings. So first, we're going to come back to the architecture that we saw together last time, so this traditional encoder/decoder architecture where you have both components. And so you have the original transformer paper from 2017 that had this architecture. But also later on, you see more architectures that built on top of it. So here we talk about the T5 family of models. So T5 is a paper that's an abbreviation of multiple Ts. So the first one is transfer, and then it's text-to-text, transformers. So this is where the T5 naming comes from. And then it's derived into multiple versions. So the T5 is like the vanilla paper. And then it had mT5. m stands for multilingual where there was some more work on the data it was trained on, as well as the vocabulary that it was computed over. And then you have ByT5, which is some of tokenizer-free method of all of this where you forego of the fact of tokenizing. And instead of that, you basically operate at the byte level. So By is like byte. And basically, you have a vocabulary size that is much smaller. So instead of having an o of 30k, you have 2 to the power of 8. So a byte is 8 bits. And then you can represent every character into bytes. So that's what they do.

OK, great. And then one thing I want to mention regarding the T5 family is that the objective function changes a bit with what the original transformer did. So the original transformer did next token prediction for the training task. But the T5 family, what they did is that they operated on the so-called span corruption task. So basically you would have your sentence as an input to the encoder. And instead of putting everything to the encoder, you would leave blanks. And this is what we call span corruption. And then the span corruption could be one or multiple tokens missing. So if I want to give an example, so for example, my teddy bear is cute and reading. So you could have my teddy bear span corrupted, and is reading. So that could be one potential encoder input. And then you could have up to n-- I mean, n is the parameterization of the number of spans that are corrupted. So you would have n such tokens here. And then the T5 families calls them sentinel tokens. So if you see sentinel tokens, they represent a span of corrupted tokens. So you have them here in the encoder. And then the decoder's work is to find each of these spans in series. So you

start with a token that denotes the first corrupted span. And then you start the decoding process until it hits a prediction that predicts the next sentinel token up to the n plus 1, 1, where the tokens that are decoded between two consecutive sentinel tokens corresponds to the corrupted spans that are recovered. So these parentheses is basically a shift from the next token prediction objective function.

Yep. Yeah. So the question is, can you elaborate with the decoding process? So what you said regarding the reconstruction is exactly right. So you have sentinel tokens that denote some missing text. So what you want to decode is that missing text. So the decoder output will be exactly like each span reconstructed. And then if you want to know how training works, you do a teacher-forcing mechanism where you input everything in the decoder, and then try to reconstruct everything at once. So any other questions? Great.

And then now we're going to talk about another class of transformers where basically you have this encoder/decoder structure, where you just forget about the decoder and then just deal with the encoder. So you might tell me, OK, hey, you cannot do generation with this. And then I would respond to you, yes, that's exactly the point. So it has encoder representations that can be used for tasks that might be more geared towards classification. So sentiment extraction, token classification, all of these things that used to be done with specific language models, they can be done with the encoder part of the transformer. And we're going to dive deeper a bit later into a three-key encoder only models. So BERT, which is like the central one, I would say in this landscape, and then two other architectures, DistilBERT and RoBERTa, that are investigating on axis of improvements. So it's going to be good to see.

So OK, great. And then there is one last class. So as Afshine just mentioned, two days LLMs, they remove the encoder part altogether. And then when you have no encoder, you don't have your encoder embeddings at the end of your stacked encoders that could be fed to the cross-attention. So this module disappears altogether. So each decoder that is stacked just has masked self-attention and an FFN as part of it. And yeah, that's basically something that has caught up since then. Because when you look at the popularity of each of these models, so you used to have these transformer-like architecture that was popular towards the beginning, where the main hypothesis was that the encoder part is very useful for getting to the decoder's representation. But

as time went, people realized that your compute budget could be best invested in the decoder only. And then there has been more investment into the kind of tasks that is, I would say, easiest to scale up and generalize-- next-word prediction-- as opposed to the task I just mentioned for T5, which might be more bespoke. So you need to corrupt things. So it's more complex. Whereas next-word prediction is, I would say, the simplest thing you can do. And it proved to be wonders and well-aligned with the task of being a helpful kind of chatbot, which is like two days' applications mainly. And then the decoder-only architectures-- so I'm not going to talk about them today. But it's going to be the central part of the next lectures when we're going to talk about LLMs more and more. OK, awesome. So now we can dive deep, as promised, into encoder-only architectures and with BERT.

BERT Deep Dive

So first we're going to start by seeing what does BERT mean. So BERT, it's an acronym that denotes Bidirectional Encoder Representations from Transformers. And we're going to see together how does each section of this acronym, what does it correspond to? So let's just start with the encoder part, which is the easiest to grasp. So as we said, we just dropped the decoder. So this encoder from transformer is basically exactly what it means. Now on the other part, so why do we talk about bidirectionality? So it's a way, so the paper's result is remarkable, because we are able from a given input to get output representations that have attended to everything for each token. And this is the case. Because since we only have the encoder, we have this self-attention layer that truly attends to every other token. And this is in contrast with the masked self-attention that you have where we said that the mask is making the attention mechanism causal. So every token can attend to itself and to the tokens before it. And this is, by the way, something that the authors discuss a lot in the paper, saying that GPT came out, GPT they're not truly bidirectional. And then these encodings that can be used for classification tasks, they truly are.

Any questions? Yep. Yep, that's right. So the question is, when you don't have the mask, each token can attend to each other. That's exactly right. And then the mask is exactly there to prevent links from tokens to those that come after them. Yeah. OK, great. So I just want to put this paper in context. And the field of NLP was booming back then. So you had another landmark paper that same

year, which was called ELMo, so Embeddings from Language Models. And I would say that the timing of that paper was a bit unfortunate, because it truly had new insights of also building bidirectional representations. But it just turns out that it came the same year as the transformer, as the kind of transformer-based same line of work. So it got a bit masked by it. So ELMo, just to give the main lines, it was based on a bidirectional LSTM where you had multiple layers, each stacked on top of each other. And basically you were able to build a bidirectional representation for each word, thanks to this architecture. And so why didn't it become as popular as BERT? It's because you had the same downsides as the previous models, where basically it's hard to scale because of this recurrence. And I think a lot of you, when you think about ELMo and BERT, you don't think about these papers at first, because they are characters in Sesame Street. And I grew up in a place where I did not know about Sesame Street. So for me, ELMo and BERT is like paper names. But it might be what represents to you at first. So I thought that was quite funny. And then researchers, they are generally quite playful. They try to fit their acronyms into themes. So if you look at paper names, you will get entertained.

OK, great. So to dive deeper into what I mentioned to be the goal of encoder-only models, so you have your set of tokens as input. And the goal here is to perform tasks that might be focused on projecting some representation somewhere, so typically classification tasks. And here, the way BERT works is very specific. So you have two kinds of tokens that you will see are structural here. So you have first the CLS token, which stands for classification. And it's basically a placeholder token that is put at the beginning of the sequence that will then, at the end of the whole attention and then projection and the whole encoder mechanism, be projected into an embedding that we can then use for classification. So CLS is just some kind of placeholder that will carry the bidirectional information of your whole input. And another token that might be useful to see is this SEP token. So it's like separator. And then you will see very soon we're going to see the objective functions that it operates on. It aims at separating two sentences. Yeah.

So this is the very high level. And then so one thing that is very interesting to see with this model is that you have this concept of multi-stage training. So you don't train the model in one shot. You do it in multiple stages. So the first stage is aimed at being aligned with the task of interest. And so this is what we call pre-training. And this pre-training, we will see in detail, is done with two

objective functions that are respectively MLM and NSP. So MLM stands for Masked Language Model. And NSP is the Next Sentence Prediction. So masked language model, it's a way for the model to learn the internal structure of the inputs. And then NSP, it might be seen as a way to see if the ordering of sentences makes sense. So we're going to go a bit deeper into each, but it's a combination of objective function that the authors assumed to be helpful for learning general embeddings of high quality. OK. So the second thing that I want to mention is that, once you have all of this, you have a further stage where you keep the embeddings that you have learned. And then you attach to it some other network, like typically a linear projection, to then fine tune the embeddings that you have learned into some target task. So this is what we call fine tuning.

Yep. So yeah. So the question is, do we still have just an encoder here? Because a next-sentence prediction could be seen as a decoder task. So actually I'm going to go in detail. The next-sentence prediction task is actually us putting two sentences, one after the other, and predicting whether they are truly consecutive. So it's a classification task. Yeah, great point. And yeah, we're going to see that in detail very soon. So before we see that in detail, I'm going to discuss the pros and cons that this method usually has. So regarding on the pro side, this pre-training mechanism can be done on fairly unlabeled data. So you still have this next-sentence prediction task. But this is something that you control, because you know which sentences follow each other. So it's something that you know it's self-supervised. And the masked language model task, we're going to see how it consists. But it's also something that is unsupervised. So this is a very interesting way to learn interesting embeddings out of unlabeled data. And then in practice is that we see that these unlabeled data leads to helpful representations being learned. And then you need very little data to build on top of it. So at the fine tuning stage, you just start from these very nice embeddings that you have learned, and you just have a few weights to tune. And then this usually leads to a performance that exceeds state of the art back then. And then regarding the downsides, we have of course all the text generation tasks that are out of reach because we don't have a decoder. And also, we can see these two stage process and the fact that we need to further tune embeddings as something that can be over hurdled. So it might be, when you compare this kind of methodology with respect to more traditional methods, that can be one shot. This could be seen

as a downside. OK, great. And I have some names regarding variants here. We're going to dive deep into two of them a bit later on.

OK. So I want us to focus on the original transformer. And we're going to go step by step into what happened to it, and then how we get the BERT architecture. So this is what we had in the original transformer paper that we saw last week. And what BERT did is extract the encoder part of it and attach basically these new objective functions that I just mentioned, alongside some new set of tricks when it comes to representing tokens. So I think we're going to go through them one by one. So first of all, one interesting note is that it uses a specific tokenizer called WordPiece. So you can see it as a tokenizer that learns on your training set, based on merge rules that maximize the likelihood. So basically, you have some huge training set, and you train a tokenizer that merges atomic tokens together to build your target vocabulary. And some order of magnitude, I mentioned o of 30k. So it's typically, I think this is the size that they chose in this paper. And in general, like vocabulary sizes in these sorts of paper are in the order of magnitude of 10 to the power of 4, 10 to the power of 5, except for our token-free method that we saw, like ByT5, which is like this very restricted set of tokens, like 2 to the power of 8, which is 256. Apart from this specific case, you always have this order of magnitude. And basically, we are going to make use of the tokens that I mentioned. Like at the beginning, so you have the CLS token that is going to carry the bidirectional representation of all your sequence. And then you have SEP tokens to separate the two sentences towards the next-sentence prediction task. And there is something that I haven't talked about just yet. So I think maybe we can talk about it at the time of the deep dive. So in order to have this masked language model task, so of course you need to mask some tokens. So we're going to see the technique that is basically applying these sorts of mask and where and at which frequency. And we're going to see with respect to the output what is going to be our task based on the resulting representation. So I'm just going to pass quickly on it here, but we're going to come back very soon.

OK, great. So one big piece of news regarding your input embeddings-- so what stays the same? So you still have these huge dictionary lookup of embeddings for each token that you're going to learn. And you're going to additively add the positional encoding that we saw at last lecture. And Afshine talked about earlier in this lecture, which can either be hard-coded or learned. I think the authors here just used a hard-coded version here, but I'm not too

sure. But it's roughly the same performance usually. But there is something new. There is the introduction of a new kind of encoding called segment encoding that is going to still be additively added to tokens, with the exception that we just have two possible encodings there. So you have segment A that represents the first sentence, and then segment B that represents the second sentence. And it's supposed to help with the NSP task to represent what could be features that can helpfully represent a sentence that precedes another one. So at least that was the hypothesis admitted by the authors. We're going to see it has been challenged later on. But this is one of the key concepts that was introduced here.

Any questions so far? Yep. Yep, so a great point. So the question is, what does segment encoding do at all? So it's basically something that is learned. So you have two indices, two basically embeddings that you can learn. You just additively add them, segment A for tokens in the first part of the sentence, and then segment B in the second part. And you just learn it with gradient descent. So you do nothing with it. I think there was another question here? Yeah. That's right. So the question is, is every token in the same sentence going to have the same segment encoding? Yes, yeah.

OK, awesome. So I'm seeing I might need to speed up a tiny bit. So yeah, the second thing that I want to highlight here is that we take the encoder part of the transformer. And nothing new here. It's just the same. We have self-attention, followed by these FFN. So this is where you're going to get your bidirectional nature of your encodings. And then basically we're assuming that training both MLM and NSP on it is going to help us learn helpful projection matrices in this encoder that can be suited for any classification task. OK, great. And as promised, I talked about detailing more about what this MLM task was going to do. So when you look at the input, you're going to basically have your input sentence, and replace at random some tokens. So it will be replaced either with the token mask. So it's in 80% of the time. In 10% of the time, the tokens selected for this MLM objective function are not going to be replaced at all. So we're just saying, it's the same token, just predict the same token. And then 10% of the time, it's going to be changed to some random other word. So at the end of it, you have the subset of tokens you're going to perform your MLM task on. And then this is how it's composed.

OK, great. And so basically the intuition here is that when you want to predict what a token is, you need to know about its context. So we're going to force the model to learn about what surrounds it, left and right. So this is the bidirectional property of this architecture put into action. OK, great. And now we're going to talk about the next-sentence prediction task where basically the process here is to select two sentences from a given corpus, and then present them side by side in order 50% of the time, and in some random order the other part of the time. And the goal is for some classification head on top of the CLS token to determine whether A and B are consecutive. So that's all it does. And so the assumption is that it also helps learn some useful embeddings.

OK, great. And here I'm going to present the notation that is used in the paper. So the paper is something I recommend reading. It's a very nice read, alongside the "Attention is All You Need." I would say it's a landmark paper. I think it has a 170k citations, something really impressive. And basically what we called n in the original transformer paper is now called L. H, that was called d model, is the dimension of our embeddings. And A is the number of attention heads, which was called the little h. So here I'm just showing these new notations for information, just to give you the mapping. But of course, together with Afshine, we're going to stay consistent with the original notations we had, so just like for information. And one interesting note is that you will see that the BERT model, when you look at some repository of models such as Hugging Face, so usually it's given in several versions. So you will see sometimes like case, uncased. So this denotes what kind of preprocessing was done to the data, whether you only have lowercase words in there or whether casing matters. So based on your task of interest, it might be one thing you want to choose from. OK, great. And I give some orders of magnitude from the paper on what values were chosen for each of these. So if I remember correctly, I think the original transformer had 12 stacked encoders and decoders. So I think some numbers here were taken from there. So yeah, order of magnitude is 100 million parameters. Any questions so far? OK, great.

BERT Fine-Tuning

And then now we're going to talk about the fine-tuning stage where the goal is to basically take whatever we have learned at pre-training stage, and then freeze those weights. And instead of training again on those same weights, you have some classification linear layer that is put on top of either the CLS token

or on top of tokens of interest. And you're going to learn about the linear embeddings there. So you're going to have some classification task. So you could either freeze all these pre-trained weights and just train on these small weights. Or you can just retrain the whole thing. I think there is multiple classification schemes. And some of it will be influenced by your willingness to retrain a lot of the network and how different the classification task is with respect to the original pre-training task. OK, great. And just to give some examples of what could be fine-tuning tasks for tasks of interest, you can have a sentiment extraction where you build a classification layer on top of the CLS token. And we give some other example question answering where basically you are given some input. And the goal of the model is to detect beginning and end spans of the response. So it's basically some objective function that is at the token level.

OK, great. So now I propose that we do some deep dive into one specific example. Our favorite example, this teddy bear is so cute. Let me just see how BERT works in practice. So basically what you would do in an uncased setting is take the sentence, pre-process it, just put everything lowercase. Then you apply the WordPiece algorithm on whatever tokenization mechanism that your tokenizer has learned. So for example here, apparently it had all these merge rules. So this is the tokens that appear in the vocabulary. And then as I promised, we add the CLS token at the beginning. And then we add the SEP token. And you also have this PAD tokens that are basically used to fill out the sequence until the end. Because when you train, you train by batch, and then batches are composed of matrices that have a fixed length.

Yeah. So let me just have this deep dive a bit the same as what we had in the transformer deep dive last time. So you have the embeddings that are learned. So it's some huge lookup table between the index of each token and the learned representation. So you add to it the position embedding of the token. And something that's new here-- the segment embedding. So exactly as we said, it's an embedding that is added to each token. And the same embedding is added to the same tokens. So all the tokens have the same segment A. Then some other embedding B is added to all the tokens of the segment B. OK, great. So now you have, similarly as the transformer, something that is position-aware and context-aware-- so not context-aware just yet, but segment-aware here. OK, great. And basically it goes through this encoder architecture. And then at the end of it, for the case of sentiment extraction, we do not care about the

embeddings that are corresponding to each token that is not the CLS token. Because what we care about is the output embedding of the CLS token, where we plug a linear layer that will learn some classification task.

So any question here? So does the fact that we drop all the output embeddings, apart from the CLS token, make sense? Yeah, yep. Yeah, so the question is, what does this FFN, is that right? What does this FFN correspond to? So basically you have a map between the dimension of the output embedding and your task of interest. So it might be classification of either positive or negative. So you have some hidden layer with some length. So you have typically two matrices to learn the projection from one to that hidden layer, and then the hidden layer to the output. And then you learn these weights in order to do the classification task, based on the embeddings that it has learned here. Yeah, great question. So that was the question I was waiting for. So the question is, why do we throw away all these other output embeddings? So we don't need them here. So we are in a classification task. We have put as convention to operate on top of some token, like the CLS token. And then the magic of this token is that all these self-attention mechanisms that are in the encoder has mixed the representation of each of the other tokens in that representation, such that the output embeddings, they are context-aware. So basically, it's an embedding that is geared towards classification. And this is what we say will be what we plug in to that linear layer. And so when I say we throw out all the other embeddings, it's actually something that we do for the case of classification. But if we do classification at the token level, then each of them may be used. So for example, when I say question answering, if you want to detect whether a given token is the beginning of an answer, beginning of end of an answer, you would typically have two FFNs that respectively predict the start and end of the answer. And you would apply it on each of these embeddings.

Any other questions? Yep. So the question is, what is query, key, and value for CLS? So it's going to be the same process as all the other tokens. So you learn here a representation for the embedding, for the token CLS. It gets projected to query. It gets projected to key. It gets projected to value. It does all its attention computation. And at the end, you get this embedding that basically that has attended to all the other ones. So it's like the answer is, it's the same as for the other tokens. So, yeah. So just treat it as one token, it could be any token, and it's the same.

OK, awesome. So I see we have five more minutes. And I'm going to quickly go through the end of it. And one thing that was great with BERT, and it was also great with ELMo, is that you have embeddings that are contextual. And here you can see that it's very easy to learn any classification task that you want, based on these learn embeddings. So this is a flexibility that was greatly appreciated here. And it's widely used in the industry. So anything that comes with sentiment detection or other classification-related tasks, it's very common to use a BERT-like model nowadays.

Extensions of BERT

And I'm going to talk a bit about its limitations now. So as you see in the original paper, I think the context length was of size 512. So it was typically limited in this early paper. Afshine has mentioned of techniques as to how we can further grow this context size without making the complexity go completely, basically like hike. And then you have some approximation methods where you compute local attention, and so on. And then the use of these tricks helps you grow the size of the context while staying within reasonable computation requirement bounds. And I'm going to talk about two other limitations that we're going to see what other models try to remedy. So there is one where the latency could be seen as high. You have a still 110 million parameters for a BERT base, so it's quite a lot. Is there a way to make this smaller and faster? And then the second one is basically you have these two objective functions of MLM and NSP. Are those two truly helpful? Is there any way we can simplify the pre-training process? So we're going to see in a second how.

So first, regarding this second limitation that we had, this basically sensitivity to cost. So who here has heard of distillation? Good, few people. So I just want to flash this quote from Hinton, Vinyals, and Jeff Dean, which I think has been informative. And it's a good mindset to have to see that the distribution that is output by a given model is actually super helpful to know what it has learned. So the soft targets contain almost all the knowledge. So it's some lecture from them that contain this quote. So basically they were at the origin of the concept of distillation later on. And you quickly see in practice that basically learning the distribution of a model is more helpful than directly learning the hard labels. So you have this concept of teacher and student models, where basically the goal in distillation is going to be to map this output distribution of a smaller model directly to your more complex model, rather than the hard labels. And the

objective function that you use for this to minimize this is the KL divergence, which basically says in a world that is described by the teacher, the teacher T, how bad is it to model with the student S. So it basically tries to assess how close the student distribution is going to be to the world's T. And one interesting note that you will see here is that you find the cross-entropy loss if your yT distribution is just a hard label. So if you have just a 1 at one position and then 0 everywhere else, you have minus log of yS, which is very interesting.

And then what the DistilBERT folks have done-- so by the way, it's a very clean paper, it's like four pages, but super impactful. So it says that if you reduce the number of layers by 2, you have a lot of gains and almost the same performance, which is very remarkable. And they basically used distillation as a way to retain that performance. So this has been like the key, alongside diminishing the number of layers.

And last, I'm going to talk about RoBERTa. That studied the fact that removing the NSP objective function led to no decrease in performance almost. So they just dropped it. And they added some tricks, such as dynamically doing the masking. So for example, for a given piece of text, at each epoch, so every time you see the same kind of text, you would change the masking. And they also had some data strategy where they saw that the model was vastly undertrained. So they increased the diversity and the size of the data lot. And they saw that benchmark performance increased quite a bit on the same benchmarks.

And that's it for today. Thank you, everyone. Have a great weekend.